arXiv:2404.05695v2 [cs.RO] 18 May 2024

Humanoid-Gym: Reinforcement Learning for Humanoid Robot with Zero-Shot Sim2Real Transfer

Xinyang Gu[2*], Yen-Jen Wang[13*], Jianyu Chen[123]

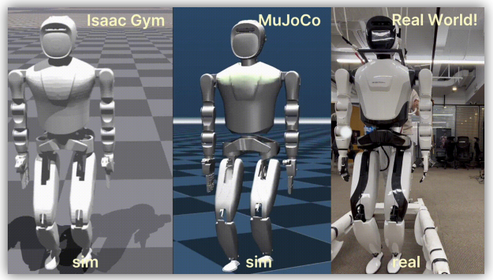
***Abstract*— Humanoid-Gym is an easy-to-use reinforcement learning (RL) framework based on Nvidia Isaac Gym, designed to train locomotion skills for humanoid robots, emphasizing zero-shot transfer from simulation to the real-world environment. Humanoid-Gym also integrates a sim-to-sim framework from Isaac Gym to Mujoco that allows users to verify the trained policies in different physical simulations to ensure the robustness and generalization of the policies. This framework is verified by RobotEra's XBot-S (1.2-meter tall humanoid robot) and XBot-L (1.65-meter tall humanoid robot) in a real-world environment with zero-shot sim-to-real transfer. The project website and source code can be found at: sites.google.com/view/humanoid-gym.**

I. INTRODUCTION

Modern environments are primarily designed for humans. Therefore, humanoid robots, with their human-like skeletal structure, are especially suited for tasks in human-centric environments, offering unique advantages over other types of robots. Recently, massively parallel deep reinforcement learning (RL) in simulation has become popular [1], [2], [3]. However, due to the complex structure of humanoid robots, the sim-to-real gap [4], [5], [6], [7] exists and is larger than that of quadrupedal robots. Therefore, we release Humanoid-Gym, an easy-to-use RL framework based on Nvidia Isaac Gym [8], designed to train locomotion skills for humanoid robots, emphasizing zero-shot transfer from simulation to the real-world environment. Humanoid-Gym features specialized rewards and domain randomization techniques for humanoid robots, simplifying the difficulty of sim-to-real transfer. Furthermore, it also integrates a sim-to-sim framework from Isaac Gym [8] to MuJoCo [9] that allows users to verify the trained policies in different physical simulations to ensure the robustness and generalization of the policies, shown in Fig. 1. Currently, Humanoid-Gym is verified by multiple humanoid robots with different sizes in a real-world environment with zero-shot sim-to-real transfer, including RobotEra's XBot-S (1.2-meter tall humanoid robot) and XBot-L (1.65-meter tall humanoid robot) [10]. The contribution of Humanoid-Gym can be summarized as follows:

- We launch an open-source reinforcement learning (RL) framework with meticulous system design.
- Our framework enables zero-shot transfer from simulation to the real world, which has been rigorously tested across humanoid robots of various sizes.

*Equal contribution. Listed alphabetically.
[1]Shanghai Qi Zhi Institute, Shanghai, China.
[2]RobotEra TECHNOLOGY CO., LTD., Beijing, China
[3]Tsinghua University, Beijing, China.



(a) Different Physical Environments

(b) Zero-Shot Sim-to-Real Transfer

Fig. 1: Humanoid-Gym enables users to train their policies within Nvidia Isaac Gym and validate them in MuJoCo. Additionally, we have successfully tested the complete pipeline with two humanoid robots. They were trained in Humanoid-Gym and transferred to real-world environments in a zero-shot manner.

- Our open-source library features a sim-to-sim validation tool, enabling users to test their policies across diverse environmental dynamics rigorously.

II. RELATED WORKS

A. Robot Learning on Locomotion Tasks

Reinforcement learning (RL) has shown promise in enabling robots to achieve stable locomotion [6], [11], [12]. Compared to prior RL efforts with quadrupedal robots [1], [13] and bipedal robots like Cassie [14], [15], our work with humanoid robots introduces a more challenging scenario for robot control. Recent studies [16], [17] have applied transformer architecture to improve the walking performance of humanoid robots on flat surfaces. Beyond lower-body control, some works [18], [19] have also explored more complex upper-body skills for humanoid robot control. However,

arXiv:2404.05695v2 [cs.RO] 18 May 2024

Humanoid-Gym：面向人形机器人的零样本***Sim2Real***迁移强化学习

徐阳[2*]，王寅仁[13*]，陈建宇[123]

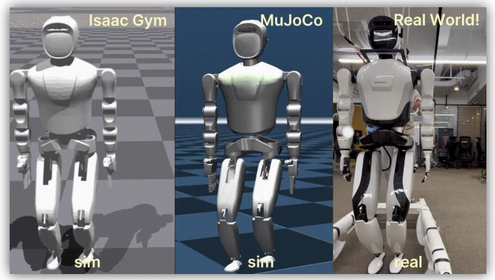
摘要***—Humanoid-Gym***是一个基于***Nvidia Isaac Gym***的易用强化学习（***RL***）框架，旨在训练人形机器人的运动技能，强调从模拟到真实世界环境的零样本迁移。***Humanoid-Gym*** 还集成了从***Isaac Gym***到***Mujoco***的***Sim-to-Sim***框架，允许用户在不同的物理模拟中验证训练的策略，以确保策略的鲁棒性和泛化能力。该框架通过在真实环境中使用零样本***Sim2Real*** 迁移验证了***RobotEra***的***XBot-S***（***1.2***米高人形机器人）和 ***XBot-L***（***1.65***米高人形机器人）。项目网站和源代码可在：***sites.google.com/view/humanoid-gym***找到。

I. 引言

现代环境主要为人设计。因此，具有类人骨骼结构的人形机器人特别适合在以人为中心的环境中执行任务，并比其他类型的机器人具有独特优势。最近，在模拟中进行的大规模并行深度强化学习（RL）变得流行 [1], [2], [3]。然而，由于人形机器人的结构复杂，模拟到现实的差距 [4], [5], [6], [7]存在，并且比四足机器人的差距更大。因此，我们发布了 Humanoid-Gym，这是一个基于 Nvidia Isaac Gym [8], 的易于使用的 RL 框架，旨在训练人形机器人的运动技能，强调从模拟到现实世界的零样本迁移。Humanoid-Gym 具有针对人形机器人的专用奖励和领域随机化技术，简化了模拟到现实的迁移难度。此外，它还集成了从 Isaac Gym [8] 到 MuJoCo [9] 的模拟到模拟框架，允许用户在不同的物理模拟中验证训练的策略，以确保策略的鲁棒性和泛化能力，如图 1 所示。目前，Humanoid-Gym 已由多个不同尺寸的人形机器人在现实世界环境中通过零样本模拟到现实迁移进行了验证，包括 RobotEra 的 XBot-S（1.2 米高的人形机器人）和 XBot-L（1.65 米高的人形机器人） [10]。Humanoid-Gym 的贡献可以总结如下：

- 我们发布了一个开源的强化学习（RL）框架，具有严谨的系统设计。
- 我们的框架支持从模拟到现实的无监督迁移，并在不同尺寸的人形机器人上经过了严格测试。

*同等贡献。按字母顺序列出.[1]上海启智研究所，中国上海.[2]RobotEraTECHNOLOGY CO., LTD.，中国北京[3]清华大学，中国北京。



(a) 不同的物理环境

(b) 零样本仿真到现实的迁移

图1：Humanoid-Gym使用户能够在Nvidia Isaac Gym中训练策略并在MuJoCo中验证它们。此外，我们已经使用两个人形机器人成功测试了整个流程。它们在Humanoid-Gym中训练，并以零样本的方式迁移到真实环境。

- 我们的开源库包含一个模拟到模拟的验证工具，允许用户在各种环境动态中严格测试他们的策略。

II. 相关工作

A. 机器人运动任务学习

强化学习（RL）已展现出帮助机器人实现稳定运动 [6], [11], [12]的潜力。与先前针对四足机器人 [1],[13]和Cassie等双足机器人 [14], [15], 的强化学习工作相比，我们针对人形机器人的研究为机器人控制引入了更具挑战性的场景。近期研究 [16], [17] 已将Transformer架构应用于提升人形机器人在平坦表面上的行走性能。除了下肢控制之外，一些工作 [18], [19]还探索了人形机器人控制中更复杂的上肢技能。然而，

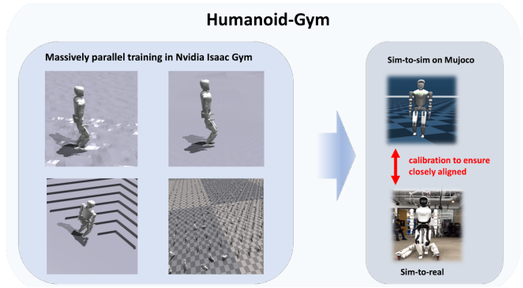


Fig. 2: Pipeline of Humanoid-Gym. Initially, we employ massively parallel deep reinforcement learning (RL) within Nvidia Isaac Gym, incorporating diverse terrains and dynamics randomization. Subsequently, we undertake sim-to-sim transfer to test policies. Due to our meticulous calibration, the performance in both MuJoCo and real-world settings aligns closely.

the sim-to-real transition for humanoid locomotion remains a significant challenge, with a notable lack of open-source resources in the robot learning community. To contribute to this area, we have developed Humanoid-Gym, an accessible framework with full codebase.

III. Method

The workflow of Humanoid-Gym is illustrated in Fig. 2. In this section, we will introduce the problem setting, system design, and reward design of our Humanoid-Gym.

A. Reinforcement Learning For Robot Control

Our approach employs a reinforcement learning model $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, T, \mathcal{O}, R, \gamma \rangle$, with $\mathcal{S}$ and $\mathcal{A}$ denoting state and action spaces, $T(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ the transition dynamics, $R(\mathbf{s}, \mathbf{a})$ the reward function, $\gamma \in [0, 1]$ the discount factor, and $\mathcal{O}$ the observation space. The framework is designed for both simulated and real-world settings, transitioning from full observability in simulations ($\mathbf{s} \in \mathcal{S}$) to partial observability in the real world ($\mathbf{o} \in \mathcal{O}$). This necessitates operating within a Partially Observable Markov Decision Process (POMDP) [20], with the policy $\pi(\mathbf{a}|\mathbf{o}_{\leq t})$ mapping observations to action distributions to maximize the expected return $J = \mathbb{E}[R_t] = \mathbb{E}\left[\sum_t \gamma^t r_t\right]$.

We leverage Proximal Policy Optimization (PPO) [21] loss, supplemented by the Asymmetric Actor Critic [22] method and the integration of privileged information during training, shifting to partial observations during deployment. The policy loss is defined as:

$$\begin{aligned}\mathcal{L}_\pi = \min \Bigg[& \frac{\pi(a_t \mid o_{\leq t})}{\pi_b(a_t \mid o_{\leq t})} A^{\pi_b}(o_{\leq t}, a_t), \\ & \text{clip}\left(\frac{\pi(a_t \mid o_{\leq t})}{\pi_b(a_t \mid o_{\leq t})}, c_1, c_2\right) A^{\pi_b}(o_{\leq t}, a_t) \Bigg]\end{aligned} \tag{1}$$

Advantage estimation utilizes Generalized Advantage Estimation (GAE) [23], requiring an updated value function:

$$\mathcal{L}_v = \|R_t - V(s_t)\|_2, \tag{2}$$

B. System design

The base poses of the robot, denoted as P^b, are six-dimensional vectors $[x, y, z, \alpha, \beta, \gamma]$, representing both the position coordinates x, y, z and the orientation angles α, β, γ in Euler notation. The joint position for each motor is represented by θ, and the corresponding joint velocity by $\dot{\theta}$. Furthermore, we define a gait phase [24], [25], which comprises two double support phases (DS) and two single support phases (SS) within each gait cycle. The cycle time, denoted as C_T, is the duration of one full gait cycle. A sinusoidal wave is employed to generate reference motion, reflecting the repetitive nature of the gait cycle involving pitch, knee, and ankle movements. Notably, we also designed a periodic stance mask $I_p(t)$ (Fig 6) that indicates foot contact patterns in synchronization with the reference motion. For instance, if the reference motion lifts the left foot, the right foot should be in the single support phase, with the foot contact mask indicated as $[0, 1]$; during DS phases, it would be $[1, 1]$.

The chosen action is the target joint position for the Proportional-Derivative (PD) controller. The policy network integrates proprioceptive sensor data, a periodic clock signal $[\sin(2\pi t/C_T), \cos(2\pi t/C_T)]$, and velocity commands $\dot{P}_{x,y,\gamma}$. A single frame of input are elaborated in Table I. Additionally, the state frame includes feet contact detect $I_d(t)$ and other privileged observations.

TABLE I: Summary of Observation Space. The table categorizes the components of the observation space into observation and state. The table also details their dimensions.

Components	Dims	Observation	State
Clock Input $(\sin(t), \cos(t))$	2	✓	✓
Commands $(\dot{P}_{x,y,\gamma})$	3	✓	✓
Joint Position (θ)	12	✓	✓
Joint Velocity $(\dot{\theta})$	12	✓	✓
Angular Velocity $(\dot{P}^b_{\alpha\beta\gamma})$	3	✓	✓
Euler Angle $(P^b_{\alpha\beta\gamma})$	3	✓	✓
Last Actions (a_{t-1})	12	✓	✓
Frictions	1		✓
Body Mass	1		✓
Base Linear Velocity	3		✓
Push Force	2		✓
Push Torques	3		✓
Tracking Difference	12		✓
Periodic Stance Mask	2		✓
Feet Contact detection	2		✓

Our control policy operates at a high frequency of 100Hz, providing enhanced granularity and precision beyond standard RL locomotion approaches. The internal PD controller runs at an even higher frequency of 1000Hz. For training simulations, Isaac Gym is utilized [8], while MuJoCo, known for its accurate physical dynamics, is chosen for sim2sim validation. This approach combines the benefits of high-speed GPU-based parallel simulation, albeit with less accuracy, and the high accuracy but slower CPU-based simulation.

The detailed settings for both algorithms and the environment designed are shown in Appendix TABLE II. We

Humanoid-Gym
Massively parallel training in Nvidia Isaac Gym
Sim-to-sim on Mujoco
calibration to ensure closely aligned
Sim-to-real
Fig. 2: Pipeline of Humanoid-Gym. Initially, we employ massively parallel deep reinforcement learning (RL) within Nvidia Isaac Gym, incorporating diverse terrains and dynamics randomization. Subsequently, we undertake sim-to-sim transfer to test policies. Due to our meticulous calibration, the performance in both MuJoCo and real-world settings aligns closely.
the sim-to-real transition for humanoid locomotion remains a significant challenge, with a notable lack of open-source resources in the robot learning community. To contribute to this area, we have developed Humanoid-Gym, an accessible framework with full codebase.
III. Method
The workflow of Humanoid-Gym is illustrated in Fig. 2. In this section, we will introduce the problem setting, system design, and reward design of our Humanoid-Gym.
A. Reinforcement Learning For Robot Control
Our approach employs a reinforcement learning model $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, T, \mathcal{O}, R, \gamma \rangle$, with $\mathcal{S}$ and $\mathcal{A}$ denoting state and action spaces, $T(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ the transition dynamics, $R(\mathbf{s}, \mathbf{a})$ the reward function, $\gamma \in [0, 1]$ the discount factor, and $\mathcal{O}$ the observation space. The framework is designed for both simulated and real-world settings, transitioning from full observability in simulations ($\mathbf{s} \in \mathcal{S}$) to partial observability in the real world ($\mathbf{o} \in \mathcal{O}$). This necessitates operating within a Partially Observable Markov Decision Process (POMDP) [20], with the policy $\pi(\mathbf{a}|\mathbf{o}_{\leq t})$ mapping observations to action distributions to maximize the expected return $J = \mathbb{E}[R_t] = \mathbb{E}\left[\sum_t \gamma^t r_t\right]$.
We leverage Proximal Policy Optimization (PPO) [21] loss, supplemented by the Asymmetric Actor Critic [22] method and the integration of privileged information during training, shifting to partial observations during deployment. The policy loss is defined as:
$$\begin{aligned}\mathcal{L}_\pi = \min \Bigg[& \frac{\pi(a_t \mid o_{\leq t})}{\pi_b(a_t \mid o_{\leq t})} A^{\pi_b}(o_{\leq t}, a_t), \\ & \text{clip}\left(\frac{\pi(a_t \mid o_{\leq t})}{\pi_b(a_t \mid o_{\leq t})}, c_1, c_2\right) A^{\pi_b}(o_{\leq t}, a_t) \Bigg]\end{aligned} \tag{1}$$
Advantage estimation utilizes Generalized Advantage Estimation (GAE) [23], requiring an updated value function:
$$\mathcal{L}_v = \|R_t - V(s_t)\|_2, \tag{2}$$
B. System design
The base poses of the robot, denoted as P^b, are six-dimensional vectors $[x, y, z, \alpha, \beta, \gamma]$, representing both the position coordinates x, y, z and the orientation angles α, β, γ in Euler notation. The joint position for each motor is represented by θ, and the corresponding joint velocity by $\dot{\theta}$. Furthermore, we define a gait phase [24], [25], which comprises two double support phases (DS) and two single support phases (SS) within each gait cycle. The cycle time, denoted as C_T, is the duration of one full gait cycle. A sinusoidal wave is employed to generate reference motion, reflecting the repetitive nature of the gait cycle involving pitch, knee, and ankle movements. Notably, we also designed a periodic stance mask $I_p(t)$ (Fig 6) that indicates foot contact patterns in synchronization with the reference motion. For instance, if the reference motion lifts the left foot, the right foot should be in the single support phase, with the foot contact mask indicated as $[0, 1]$; during DS phases, it would be $[1, 1]$.
The chosen action is the target joint position for the Proportional-Derivative (PD) controller. The policy network integrates proprioceptive sensor data, a periodic clock signal $[\sin(2\pi t/C_T), \cos(2\pi t/C_T)]$, and velocity commands $\dot{P}_{x,y,\gamma}$. A single frame of input are elaborated in Table I. Additionally, the state frame includes feet contact detect $I_d(t)$ and other privileged observations.
TABLE I: Summary of Observation Space. The table categorizes the components of the observation space into observation and state. The table also details their dimensions.
Components | Dims | Observation | State
Clock Input $(\sin(t), \cos(t))$ | 2 | ✓ | ✓
Commands $(\dot{P}_{x,y,\gamma})$ | 3 | ✓ | ✓
Joint Position (θ) | 12 | ✓ | ✓
Joint Velocity $(\dot{\theta})$ | 12 | ✓ | ✓
Angular Velocity $(\dot{P}^b_{\alpha\beta\gamma})$ | 3 | ✓ | ✓
Euler Angle $(P^b_{\alpha\beta\gamma})$ | 3 | ✓ | ✓
Last Actions (a_{t-1}) | 12 | ✓ | ✓
Frictions | 1 | | ✓
Body Mass | 1 | | ✓
Base Linear Velocity | 3 | | ✓
Push Force | 2 | | ✓
Push Torques | 3 | | ✓
Tracking Difference | 12 | | ✓
Periodic Stance Mask | 2 | | ✓
Feet Contact detection | 2 | | ✓
Our control policy operates at a high frequency of 100Hz, providing enhanced granularity and precision beyond standard RL locomotion approaches. The internal PD controller runs at an even higher frequency of 1000Hz. For training simulations, Isaac Gym is utilized [8], while MuJoCo, known for its accurate physical dynamics, is chosen for sim2sim validation. This approach combines the benefits of high-speed GPU-based parallel simulation, albeit with less accuracy, and the high accuracy but slower CPU-based simulation.
The detailed settings for both algorithms and the environment designed are shown in Appendix TABLE II. We

use multi-frames of observations and privilege observation, which is crucial for locomotion tasks on uneven terrain.

C. Reward Design

Our reward function directs the robot to adhere to velocity commands, sustain a stable gait, and achieve smooth contact. The reward function is structured into four key components: (1) velocity tracking, (2) gait reward, and (3) regularization terms.

The reward function is summarized in Appendix Table IV. It is important to note that the commands $\text{CMD}_{z,\gamma,\beta}$ (velocity mismatch term) are intentionally set to zero. This is because we do not control them; rather, we aim to maintain their values at zero to ensure stable and smooth walking. In addition, the reward (contact pattern) encourages feet to align with their contact masks, denoting swing, and stance phases, as illustrated in Appendix Fig. 6. Therefore, the total reward at any time step t is computed as the weighted sum of individual reward components, expressed as $r_t = \sum_i r_i \cdot \mu_i$, where μ_i represents the weighting factor for each reward component r_i.

IV. Experiments

In this section, we will illustrate the result of zero-shot transfer for both sim-to-sim and sim-to-real scenarios. Additionally, we also provide visualization of the calibration for MuJoCo to verify the effectiveness of sim-to-sim. For the validation, we utilized Robot Era's humanoid robots, XBot-S and XBot-L, measuring 1.2 meters and 1.65 meters in height, shown in Appendix Fig. 5, respectively.

A. Zero-shot Transfer

We carefully design domain randomization terms, as detailed in Appendix TABLE III, to minimize the sim2real gap, following the approach outlined in [26]. Our agents are capable of transitioning to real-world environments via zero-shot sim-to-real transfer, which is illustrated in Fig. 1. The standard procedure involves training agents on GPUs, followed by policy analysis in MuJoCo. For a comprehensive evaluation, we developed two types of terrains: flat and uneven, as depicted in Appendix Fig. 7. The flat terrain replicates the environment encountered during training in Isaac Gym, while the uneven terrain offers a substantially more challenging landscape, differing significantly from our initial training scenarios. Remarkably, our trained policies enable the robots to traverse both types of terrain successfully.

B. Calibration for MuJoCo

We meticulously calibrated the MuJoCo environment to align its dynamics and performance more closely with that of the real world. By comparing the leg swing sine waves in both the MuJoCo and the real-world environments, which represent joint positions over time that track a specified sine wave, we observed nearly identical trajectories. These are depicted in Fig. 3. Furthermore, we also compare the resulting phase portrait of the left knee joint and left ankle pitch joint[17] within 5-second trajectories, 0.5 n/s heading velocity, as shown in Fig. 4. It is clear to see that the dynamics in MuJoCo are closer to the real environment than Isaac Gym.

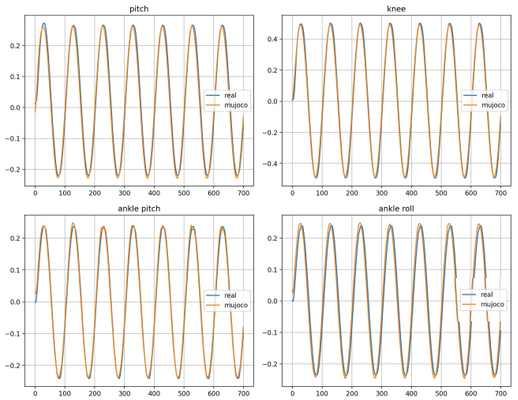


Fig. 3: Sine wave in Both MuJoCo and real-world environment. It can be found that the trajectories of the two are very close after calibration.

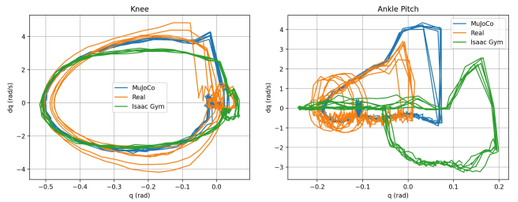


Fig. 4: Phase Portrait for MuJoCo, Real-World Environment, and Isaac Gym.

V. Conclusions

Humanoid-Gym facilitates zero-shot transfer for humanoid robots of two distinct sizes, from sim-to-sim and sim-to-real, via a specialized reward function tailored for humanoid robotics. Our experimental outcomes reveal that the adjusted MuJoCo simulation closely mirrors the dynamics and performance of the real-world environment. This congruence enables researchers lacking physical robots to validate training policies through sim-to-sim, significantly enhancing the potential for successful sim-to-real transfers.

Acknowledgment

The implementation of Humanoid-Gym relies on resources from legged_gym and rsl_rl projects[1] created by the Robotic Systems Lab. We specifically utilize the 'LeggedRobot' implementation from their research to enhance our codebase.

use multi-frames of observations and privilege observation, which is crucial for locomotion tasks on uneven terrain.

C. Reward Design

Our reward function directs the robot to adhere to velocity commands, sustain a stable gait, and achieve smooth contact. The reward function is structured into four key components: (1) velocity tracking, (2) gait reward, and (3) regularization terms.

The reward function is summarized in Appendix Table IV. It is important to note that the commands $\text{CMD}_{z,\gamma,\beta}$ (velocity mismatch term) are intentionally set to zero. This is because we do not control them; rather, we aim to maintain their values at zero to ensure stable and smooth walking. In addition, the reward (contact pattern) encourages feet to align with their contact masks, denoting swing, and stance phases, as illustrated in Appendix Fig. 6. Therefore, the total reward at any time step t is computed as the weighted sum of individual reward components, expressed as $r_t = \sum_i r_i \cdot \mu_i$, where μ_i represents the weighting factor for each reward component r_i.

IV. Experiments

In this section, we will illustrate the result of zero-shot transfer for both sim-to-sim and sim-to-real scenarios. Additionally, we also provide visualization of the calibration for MuJoCo to verify the effectiveness of sim-to-sim. For the validation, we utilized Robot Era's humanoid robots, XBot-S and XBot-L, measuring 1.2 meters and 1.65 meters in height, shown in Appendix Fig. 5, respectively.

A. Zero-shot Transfer

We carefully design domain randomization terms, as detailed in Appendix TABLE III, to minimize the sim2real gap, following the approach outlined in [26]. Our agents are capable of transitioning to real-world environments via zero-shot sim-to-real transfer, which is illustrated in Fig. 1. The standard procedure involves training agents on GPUs, followed by policy analysis in MuJoCo. For a comprehensive evaluation, we developed two types of terrains: flat and uneven, as depicted in Appendix Fig. 7. The flat terrain replicates the environment encountered during training in Isaac Gym, while the uneven terrain offers a substantially more challenging landscape, differing significantly from our initial training scenarios. Remarkably, our trained policies enable the robots to traverse both types of terrain successfully.

B. Calibration for MuJoCo

We meticulously calibrated the MuJoCo environment to align its dynamics and performance more closely with that of the real world. By comparing the leg swing sine waves in both the MuJoCo and the real-world environments, which represent joint positions over time that track a specified sine wave, we observed nearly identical trajectories. These are depicted in Fig. 3. Furthermore, we also compare the resulting phase portrait of the left knee joint and left ankle pitch joint[17] within 5-second trajectories, 0.5 n/s heading velocity, as shown in Fig. 4. It is clear to see that the dynamics in MuJoCo are closer to the real environment than Isaac Gym.

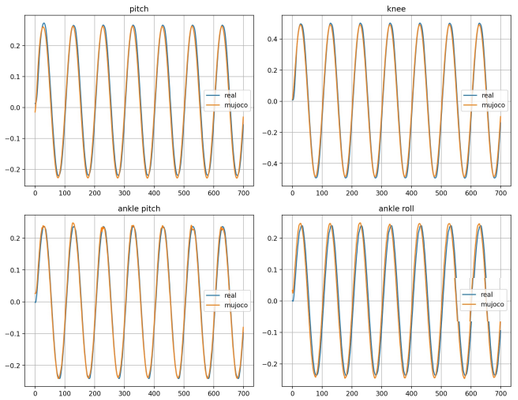


Fig. 3: Sine wave in Both MuJoCo and real-world environment. It can be found that the trajectories of the two are very close after calibration.

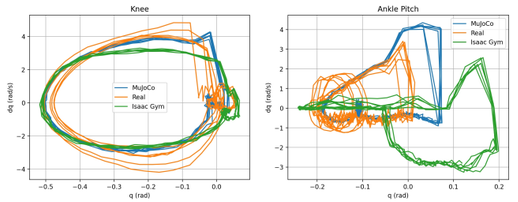


Fig. 4: Phase Portrait for MuJoCo, Real-World Environment, and Isaac Gym.

V. CONCLUSIONS

Humanoid-Gym facilitates zero-shot transfer for humanoid robots of two distinct sizes, from sim-to-sim and sim-to-real, via a specialized reward function tailored for humanoid robotics. Our experimental outcomes reveal that the adjusted MuJoCo simulation closely mirrors the dynamics and performance of the real-world environment. This congruence enables researchers lacking physical robots to validate training policies through sim-to-sim, significantly enhancing the potential for successful sim-to-real transfers.

ACKNOWLEDGMENT

The implementation of Humanoid-Gym relies on resources from legged_gym and rsl_rl projects[1] created by the Robotic Systems Lab. We specifically utilize the 'LeggedRobot' implementation from their research to enhance our codebase.

REFERENCES

[1] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Conference on Robot Learning*. PMLR, 2022, pp. 91–100.

[2] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "Rma: Rapid motor adaptation for legged robots," *arXiv preprint arXiv:2107.04034*, 2021.

[3] Y. Guo, Z. Jiang, Y.-J. Wang, J. Gao, and J. Chen, "Decentralized motor skill learning for complex robotic systems," *IEEE Robotics and Automation Letters*, 2023.

[4] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 3803–3810.

[5] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: a survey," in *2020 IEEE symposium series on computational intelligence (SSCI)*. IEEE, 2020, pp. 737–744.

[6] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots," *arXiv preprint arXiv:1804.10332*, 2018.

[7] A. Kadian, J. Truong, A. Gokaslan, A. Clegg, E. Wijmans, S. Lee, M. Savva, S. Chernova, and D. Batra, "Sim2real predictivity: Does evaluation in simulation predict real-world performance?" *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 6670–6677, 2020.

[8] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, *et al.*, "Isaac gym: High performance gpu-based physics simulation for robot learning," *arXiv preprint arXiv:2108.10470*, 2021.

[9] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 5026–5033.

[10] RobotEra, "Robotera technology co.,ltd." [Online]. Available: https://www.robotera.com/

[11] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[12] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[13] G. B. Margolis, G. Yang, K. Paigwar, T. Chen, and P. Agrawal, "Rapid locomotion via reinforcement learning," *The International Journal of Robotics Research*, p. 02783649231224053, 2022.

[14] Z. Li, X. Cheng, X. B. Peng, P. Abbeel, S. Levine, G. Berseth, and K. Sreenath, "Reinforcement learning for robust parameterized locomotion control of bipedal robots," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 2811–2817.

[15] A. Kumar, Z. Li, J. Zeng, D. Pathak, K. Sreenath, and J. Malik, "Adapting rapid motor adaptation for bipedal robots," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 1161–1168.

[16] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, "Learning humanoid locomotion with transformers," *arXiv preprint arXiv:2303.03381*, 2023.

[17] I. Radosavovic, B. Zhang, B. Shi, J. Rajasegaran, S. Kamat, T. Darrell, K. Sreenath, and J. Malik, "Humanoid locomotion as next token prediction," *arXiv preprint arXiv:2402.19469*, 2024.

[18] T. He, Z. Luo, W. Xiao, C. Zhang, K. Kitani, C. Liu, and G. Shi, "Learning human-to-humanoid real-time whole-body teleoperation," *arXiv preprint arXiv:2403.04436*, 2024.

[19] X. Cheng, Y. Ji, J. Chen, R. Yang, G. Yang, and X. Wang, "Expressive whole-body control for humanoid robots," *arXiv preprint arXiv:2402.16796*, 2024.

[20] M. T. Spaan, "Partially observable markov decision processes," in *Reinforcement learning: State-of-the-art*. Springer, 2012, pp. 387–414.

[21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[22] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, "Asymmetric actor critic for image-based robot learning," *arXiv preprint arXiv:1710.06542*, 2017.

[23] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[24] J. Siekmann, Y. Godse, A. Fern, and J. Hurst, "Sim-to-real learning of all common bipedal gaits via periodic reward composition," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 7309–7315.

[25] Y. Yang, T. Zhang, E. Coumans, J. Tan, and B. Boots, "Fast and efficient locomotion via learned gait transitions," in *Conference on robot learning*. PMLR, 2022, pp. 773–783.

[26] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.

REFERENCES

[1] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Conference on Robot Learning*. PMLR, 2022, pp. 91–100.

[2] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "Rma: Rapid motor adaptation for legged robots," *arXiv preprint arXiv:2107.04034*, 2021.

[3] Y. Guo, Z. Jiang, Y.-J. Wang, J. Gao, and J. Chen, "Decentralized motor skill learning for complex robotic systems," *IEEE Robotics and Automation Letters*, 2023.

[4] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 3803–3810.

[5] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: a survey," in *2020 IEEE symposium series on computational intelligence (SSCI)*. IEEE, 2020, pp. 737–744.

[6] J. Tan, T. Zhang, E. Coumans, A. Iscen, Y. Bai, D. Hafner, S. Bohez, and V. Vanhoucke, "Sim-to-real: Learning agile locomotion for quadruped robots," *arXiv preprint arXiv:1804.10332*, 2018.

[7] A. Kadian, J. Truong, A. Gokaslan, A. Clegg, E. Wijmans, S. Lee, M. Savva, S. Chernova, and D. Batra, "Sim2real predictivity: Does evaluation in simulation predict real-world performance?" *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 6670–6677, 2020.

[8] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, *et al.*, "Isaac gym: High performance gpu-based physics simulation for robot learning," *arXiv preprint arXiv:2108.10470*, 2021.

[9] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 5026–5033.

[10] RobotEra, "Robotera technology co.,ltd." [Online]. Available: https://www.robotera.com/

[11] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[12] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[13] G. B. Margolis, G. Yang, K. Paigwar, T. Chen, and P. Agrawal, "Rapid locomotion via reinforcement learning," *The International Journal of Robotics Research*, p. 02783649231224053, 2022.

[14] Z. Li, X. Cheng, X. B. Peng, P. Abbeel, S. Levine, G. Berseth, and K. Sreenath, "Reinforcement learning for robust parameterized locomotion control of bipedal robots," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 2811–2817.

[15] A. Kumar, Z. Li, J. Zeng, D. Pathak, K. Sreenath, and J. Malik, "Adapting rapid motor adaptation for bipedal robots," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 1161–1168.

[16] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath, "Learning humanoid locomotion with transformers," *arXiv preprint arXiv:2303.03381*, 2023.

[17] I. Radosavovic, B. Zhang, B. Shi, J. Rajasegaran, S. Kamat, T. Darrell, K. Sreenath, and J. Malik, "Humanoid locomotion as next token prediction," *arXiv preprint arXiv:2402.19469*, 2024.

[18] T. He, Z. Luo, W. Xiao, C. Zhang, K. Kitani, C. Liu, and G. Shi, "Learning human-to-humanoid real-time whole-body teleoperation," *arXiv preprint arXiv:2403.04436*, 2024.

[19] X. Cheng, Y. Ji, J. Chen, R. Yang, G. Yang, and X. Wang, "Expressive whole-body control for humanoid robots," *arXiv preprint arXiv:2402.16796*, 2024.

[20] M. T. Spaan, "Partially observable markov decision processes," in *Reinforcement learning: State-of-the-art*. Springer, 2012, pp. 387–414.

[21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[22] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, "Asymmetric actor critic for image-based robot learning," *arXiv preprint arXiv:1710.06542*, 2017.

[23] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[24] J. Siekmann, Y. Godse, A. Fern, and J. Hurst, "Sim-to-real learning of all common bipedal gaits via periodic reward composition," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 7309–7315.

[25] Y. Yang, T. Zhang, E. Coumans, J. Tan, and B. Boots, "Fast and efficient locomotion via learned gait transitions," in *Conference on robot learning*. PMLR, 2022, pp. 773–783.

[26] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.

APPENDIX

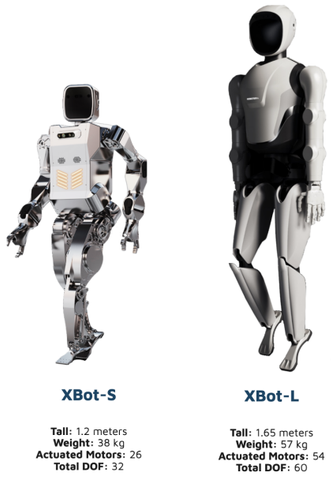


Fig. 5: Hardware Platform. Our Humanoid-Gym framework is tested on two distinct sizes of humanoid robots, XBot-S and XBot-L, provided by Robot Era.

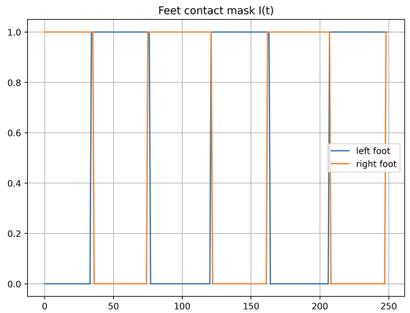


Fig. 6: The stance mask is the contact planning for the left (L) and right (R) feet, where 0 indicates the swing phase and 1 indicates the stance phase is expected.

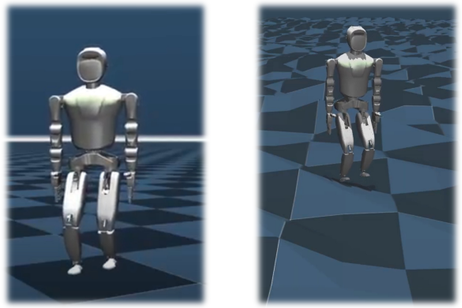


Fig. 7: Terrains in MuJoCo. Humanoid-Gym provides two types of terrains utilized for sim-to-sim validation: flat planes and uneven terrains.

TABLE II: Hyperparameters.

Parameter	Value
Number of Environments	8192
Number Training Epochs	2
Batch size	8192×24
Episode Length	2400 steps
Discount Factor	0.994
GAE discount factor	0.95
Entropy Regularization Coefficient	0.001
$c1$	0.8
$c2$	1.2
Learning rate	1e-5
Frame Stack of Single Observation	15
Frame Stack of Single Privileged Observation	3
Number of Single Observation	47
Number of Single Privileged Observation	73

TABLE III: Overview of Domain Randomization. Presented are the domain randomization terms and the associated parameter ranges. Additive randomization increments the parameter by a value within the specified range while scaling randomization adjusts it by a multiplicative factor from the same range.

Parameter	Unit	Range	Operator	Type
Joint Position	rad	[-0.05, 0.05]	additive	Gaussian (1σ)
Joint Velocity	rad/s	[-0.5, 0.5]	additive	Gaussian (1σ)
Angular Velocity	rad/s	[-0.1, 0.1]	additive	Gaussian (1σ)
Euler Angle	rad	[-0.03, 0.03]	additive	Gaussian (1σ)
System Delay	ms	[0, 10]	-	Uniform
Friction	-	[0.1, 2.0]	-	Uniform
Motor Strength	%	[95, 105]	scaling	Gaussian (1σ)
Payload	kg	[-5, 5]	additive	Gaussian (1σ)

APPENDIX

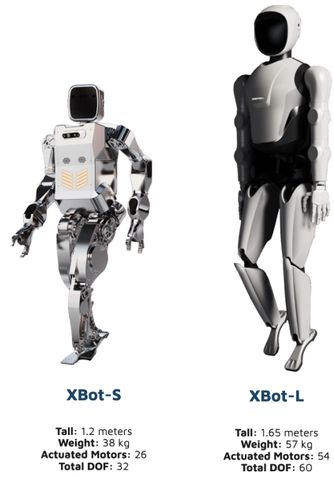


Fig. 5: Hardware Platform. Our Humanoid-Gym framework is tested on two distinct sizes of humanoid robots, XBot-S and XBot-L, provided by Robot Era.

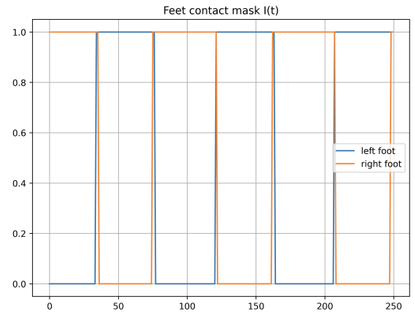


Fig. 6: The stance mask is the contact planning for the left (L) and right (R) feet, where 0 indicates the swing phase and 1 indicates the stance phase is expected.

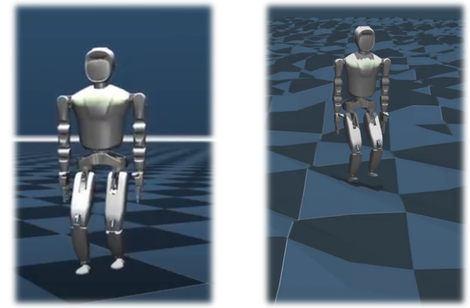


Fig. 7: Terrains in MuJoCo. Humanoid-Gym provides two types of terrains utilized for sim-to-sim validation: flat planes and uneven terrains.

TABLE II: Hyperparameters.

Parameter	Value
Number of Environments	8192
Number Training Epochs	2
Batch size	8192×24
Episode Length	2400 steps
Discount Factor	0.994
GAE discount factor	0.95
Entropy Regularization Coefficient	0.001
$c1$	0.8
$c2$	1.2
Learning rate	1e-5
Frame Stack of Single Observation	15
Frame Stack of Single Privileged Observation	3
Number of Single Observation	47
Number of Single Privileged Observation	73

TABLE III: Overview of Domain Randomization. Presented are the domain randomization terms and the associated parameter ranges. Additive randomization increments the parameter by a value within the specified range while scaling randomization adjusts it by a multiplicative factor from the same range.

Parameter	Unit	Range	Operator	Type
Joint Position	rad	[-0.05, 0.05]	additive	Gaussian (1σ)
Joint Velocity	rad/s	[-0.5, 0.5]	additive	Gaussian (1σ)
Angular Velocity	rad/s	[-0.1, 0.1]	additive	Gaussian (1σ)
Euler Angle	rad	[-0.03, 0.03]	additive	Gaussian (1σ)
System Delay	ms	[0, 10]	-	Uniform
Friction	-	[0.1, 2.0]	-	Uniform
Motor Strength	%	[95, 105]	scaling	Gaussian (1σ)
Payload	kg	[-5, 5]	additive	Gaussian (1σ)

TABLE IV: In defining the reward function, we use a tracking error metric denoted by $\phi(e, w)$. This metric is expressed as $\phi(e, w) := \exp(-w \cdot \|e\|^2)$, where e represents the tracking error, and w is the associated weight. The target base height is set to $0.7\,\text{m}$.

Reward	**Equation** (r_i)	**reward scale**(μ_i)
Lin. velocity tracking	$\phi(\dot{P}^b_{xyz} - \text{CMD}_{xyz}, 5)$	1.2
Ang. velocity tracking	$\phi(\dot{P}^b_{\alpha\beta\gamma} - \text{CMD}_{\alpha\beta\gamma}, 5)$	1.0
Orientation tracking	$\phi(P^b_{\alpha,\beta}, 5)$	1.0
Base height tracking	$\phi(P^b_z - 0.7, 100)$	0.5
Velocity mismatch	$\phi(\dot{P}^b_{z,\gamma,\beta} - \text{CMD}_{z,\gamma,\beta}, 5)$	0.5
Contact Pattern	$\phi(I_p(t) - I_d(t), \infty)$	1.0
Joint Position Tracking	$\phi(\theta - \theta_{\text{target}}, 2)$	1.5
Default Joint	$\phi(\theta_t - \theta_0, 2)$	0.2
Energy Cost	$\lvert\tau\rvert\lvert\dot{\theta}\rvert$	-0.0001
Action Smoothness	$\|a_t - 2a_{t-1} + a_{t-2}\|_2$	-0.01
Large contact	$\max(F_{L,R} - 400, 0, 100)$	-0.01

TABLE IV: In defining the reward function, we use a tracking error metric denoted by $\phi(e, w)$. This metric is expressed as $\phi(e, w) := \exp(-w \cdot \|e\|^2)$, where e represents the tracking error, and w is the associated weight. The target base height is set to $0.7\,\text{m}$.

Reward	**Equation** (r_i)	**reward scale**(μ_i)
Lin. velocity tracking	$\phi(\dot{P}^b_{xyz} - \text{CMD}_{xyz}, 5)$	1.2
Ang. velocity tracking	$\phi(\dot{P}^b_{\alpha\beta\gamma} - \text{CMD}_{\alpha\beta\gamma}, 5)$	1.0
Orientation tracking	$\phi(P^b_{\alpha,\beta}, 5)$	1.0
Base height tracking	$\phi(P^b_z - 0.7, 100)$	0.5
Velocity mismatch	$\phi(\dot{P}^b_{z,\gamma,\beta} - \text{CMD}_{z,\gamma,\beta}, 5)$	0.5
Contact Pattern	$\phi(I_p(t) - I_d(t), \infty)$	1.0
Joint Position Tracking	$\phi(\theta - \theta_{\text{target}}, 2)$	1.5
Default Joint	$\phi(\theta_t - \theta_0, 2)$	0.2
Energy Cost	$\lvert\tau\rvert\lvert\dot{\theta}\rvert$	-0.0001
Action Smoothness	$\|a_t - 2a_{t-1} + a_{t-2}\|_2$	-0.01
Large contact	$\max(F_{L,R} - 400, 0, 100)$	-0.01